

Guide de Référence : Fonctions Réseau du Serveur IRC

Ce document détaille le fonctionnement en profondeur des principales fonctions système et réseau utilisées pour la création de ton serveur IRC en C++ 98.

1. Initialisation du Serveur

1.1 socket()

```
int socket(int domain, int type, int protocol);
```

Rôle : Crée un point de terminaison (un "tuyau" réseau) pour la communication. C'est la première étape indispensable.

- **domain (AF_INET)** : Spécifie qu'on utilise le protocole IPv4.
- **type (SOCK_STREAM)** : Indique qu'on veut une connexion fiable et ordonnée (protocole TCP).
- **protocol (0)** : Laisse le système choisir le protocole par défaut associé au type (ici TCP).

Retourne un entier (File Descriptor). S'il vaut -1, la création a échoué.

1.2 setsockopt()

```
int setsockopt(int sockfd, int level, int optname, const void *optval,  
socklen_t optlen);
```

Rôle : Configure des options spécifiques pour un socket existant.

- **level (SOL_SOCKET)** : Indique que l'option s'applique au niveau du socket lui-même.
- **optname (SO_REUSEADDR)** : **Crucial pour le développement !** Permet de redémarrer le serveur immédiatement après un crash sans attendre que le port soit purgé par le système.

1.3 fcntl()

```
int fcntl(int fd, int cmd, ... /* arg */ );
```

Rôle : Modifie le comportement d'un descripteur de fichier (File Control). Dans notre cas, il rend le socket non-bloquant.

- **cmd (F_SETFL)** : Demande à définir un nouveau "drapeau" (flag) de statut.
- **arg (O_NONBLOCK)** : Configure le socket pour qu'il ne bloque jamais le programme en attendant des données. Si rien n'est reçu, les fonctions retournent immédiatement une erreur spécifique plutôt que de figer le serveur.

1.4 bind() & listen()

bind() : Associe le socket (ton tuyau) à une adresse IP spécifique (**INADDR_ANY**) et à un port (ex: 6667). C'est ce qui donne une véritable "adresse" physique à ton serveur.

listen() : Indique au système que ce socket est désormais passif, c'est-à-dire qu'il est ouvert au public et qu'il écoute les demandes de connexions entrantes (il définit la taille de la file d'attente avec **SOMAXCONN**).

2. Le Cerveau Asynchrone : Epoll

2.1 epoll_create1()

```
int epoll_create1(int flags);
```

Rôle : Crée l'instance "chef de quai" epoll. Il retourne un nouveau descripteur (**_EpollFD**) qui va servir à surveiller tous les autres.

2.2 epoll_ctl()

```
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);
```

Rôle : C'est la fonction d'enregistrement. Elle permet d'ajouter, modifier ou supprimer un socket de la liste de surveillance.

- **op (EPOLL_CTL_ADD / EPOLL_CTL_DEL)** : L'opération à effectuer.
- **event->events (EPOLLIN)** : Demande à être prévenu spécifiquement quand il y a des données à lire (IN) sur ce socket.

2.3 epoll_wait()

```
int epoll_wait(int epfd, struct epoll_event *events, int maxevents, int timeout);
```

Rôle : Met le programme en pause jusqu'à ce qu'il y ait du mouvement sur l'un des sockets surveillés. Il remplit un tableau (**events**) avec uniquement les sockets qui ont reçu quelque chose.

3. La Gestion du Flux Client

3.1 accept()

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

Rôle : Quand epoll signale qu'un nouveau client tente de se connecter sur le port du serveur, **accept** valide la connexion et génère un TOUT NOUVEAU descripteur de fichier, dédié exclusivement à la conversation avec ce client précis.

3.2 `recv()` & `send()`

`recv()` : Lit les données brutes envoyées par un client spécifique. Elle retourne le nombre d'octets lus. Si elle retourne 0, cela signifie que le client s'est déconnecté proprement. Si elle retourne -1 (et que ce n'est pas lié au mode non-bloquant), c'est une erreur réseau.

`send()` : L'inverse exact. Permet d'injecter des données (comme tes messages IRC ou tes RPL) dans le tuyau TCP d'un client. Il garantit la livraison et l'ordre des messages.